

Received 13 May 2024, accepted 27 May 2024, date of publication 5 June 2024, date of current version 13 June 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3409745

RESEARCH ARTICLE

Measuring and Improving the Energy Efficiency of Large Language Models Inference

MAURICIO FADEL ARGERICH^{ID} AND MARTA PATIÑO-MARTÍNEZ

Escuela Técnica Superior de Ingenieros Informáticos, Universidad Politécnica de Madrid, 28040 Madrid, Spain

Corresponding author: Mauricio Fadel Argerich (mauricio.fadel@alumnos.upm.es)

This work was supported in part by the Project QoSDATA PID2020-119461GB-I00 funded by MCIN/AEI/10.13039/501100011033, and in part by the Project AI4PublicPolicy funded by the European Union's Horizon 2020 Research and Innovation Program under Grant 101004480.

ABSTRACT Recent improvements in the accuracy of machine learning (ML) models in the language domain have propelled their use in a multitude of products and services, touching millions of lives daily. These new levels of accuracy have been attained mainly through exponential growth in model size, creating a new category of models known as Large Language Models (LLMs) and leading to a substantial increase in computing and energy demands. While recent studies have focused on measuring and improving the energy consumption of LLMs during training, inference has received little attention. In this article, we present an approach to profile the energy consumption of LLMs during inference and leverage it to improve energy efficiency. For this, we deploy several state-of-the-art LLMs and observe how model size, number of layers, parallelized attention, and even vocabulary size affect their energy consumption. In addition, we leverage input batch size and different quantization levels to optimize their inference energy efficiency and latency.

INDEX TERMS Energy consumption, energy efficiency, machine learning, deep learning, large language models.

I. INTRODUCTION

Machine learning (ML) is dramatically transforming the world we live in. From reading AI-generated news summaries [1], to developing software with AI assistants [2], to using AI-powered tools to generate images for business presentations [3], to enjoying personalized recommendations of movies and series [4], we interact with ML-based applications several times every day. All of these services are powered by large ML models, whose size – defined by the number of trainable parameters – has exponentially grown in recent years in the quest for better accuracy.

Research shows that since 2018, the size of ML models has doubled every 8 months, mainly driven by models in the language domain [5]. We can observe this trend in the transformer-based language models introduced by OpenAI in the past few years: the first GPT model introduced in 2018 had 117 million parameters; its continuation GPT-2, released in 2019, had 1.7 billion parameters; GPT-3, released

just one year later had 175 billion parameters [6]; and GPT-4, released in 2023, is estimated to have more than a trillion parameters [7]. Due to these staggering sizes, the latest language models, including the GPT family, have become popularly known as Large Language Models (LLMs.)

This growth in size has caused a dramatic increase in the computing and electric power required for training and inference, and consequently, a higher environmental impact due to CO₂ emissions. For instance, paper [8] estimated that training GPT-3 required 1287 MWh, equivalent to the mean yearly energy consumption of 420 persons [9]. Energy consumption of LLMs has recently gained attention in both the academic community [10], [11] and industry [12]. However, the focus is usually put on tracking and reducing the energy – and corresponding CO₂ emissions – used for training models while disregarding their energy consumption during inference.

While the energy consumption during inference is less intensive than during training, we argue that energy consumption during inference is more important for a number of reasons. First, models are generally trained only once while

The associate editor coordinating the review of this manuscript and approving it for publication was Mouloud Denai^{ID}.

TABLE 1. List of abbreviations.

Abbreviation	Definition
AI	Artificial Intelligence
CNN	Convolutional Neural Network
CO ₂	Carbon Dioxide
CPU	Central Processing Unit
DRAM	Dynamic Random-Access Memory
eBPF	Extended Berkeley Packet Filter
FP16	16-bit Floating-Point Number
GPU	Graphics Processing Unit
GPT	Generative Pre-trained Transformer
INT8	8-bit integer
Intel RAPL	Intel Running Average Power Limit
IO	Input/Output
LLM	Large Language Model
ML	Machine Learning
NVIDIA NVML	NVIDIA Management Library
NVIDIA-SMI	NVIDIA System Management Interface
PUE	Power Usage Effectiveness
QA	Question Answering
SSD	Solid-State Drive
Wh (and MWh)	Watt-hour (and Megawatt-hour)

they are used for inference thousands if not millions of times. For instance, ChatGPT – a chatbot based on GPT-3 – was reported to have 13 million daily visitors during January 2023 [13]. Assuming the model was queried only once per visit, this translates into more than 400 million queries in just one month. Based on the reported average consumption of a similar-sized model called Bloom, which consumes approximately 3.96 Wh per request [14], we project that ChatGPT energy consumption was over 1500 MWh during January 2023. This is already higher than the energy used for its training. Secondly, training can be deferred to times when energy is generated from a higher proportion of low-carbon or renewable sources, reducing the CO₂ emissions related to the model training while inference is usually time-sensitive and its execution cannot be delayed. Finally, training can also be run on low-carbon intensity geographical regions, while on the contrary, the location where the model is deployed for inference is usually conditioned by different requirements such as latency, network bandwidth, data privacy and legislation.

In this paper, we profile the energy consumption of LLMs to obtain insights to improve their energy consumption when deploying and running inferences on these models. We focus our attention on LLMs because their energy consumption is significantly higher than that of traditional software and other ML models due to two main factors: (1) LLMs represent most of the largest models currently in use [5], computing billions of operations to answer every request they receive, and (2) LLMs have millions of daily requests and end users through applications such as ChatGPT [15], Microsoft 365 Copilot [16], and others. Our contributions are the following:

- 1) We analyze the current challenges of monitoring the energy consumption of software, and in particular, LLMs, in Section II.

- 2) We design and implement a software tool to easily measure the energy consumption of any code on Linux, capable of distinguishing the consumption of CPU, memory, GPU and storage (Section IV). To the best of our knowledge, this is the first pure software tool to measure the energy consumption of the storage on Linux.
- 3) We deploy multiple, state-of-the-art LLMs and observe how model size, architecture, batch size, and weights quantization affect their inference energy consumption and latency in Sections IV and VI.
- 4) We provide several insights learnt which will be useful to monitor and reduce the energy consumption of LLMs in Section VII.

The rest of the paper is organized as follows. Section II introduces the basics and challenges of measuring the energy consumption of software and LLMs. Related work is described in Section III. Section IV describes the design and implementation of an energy meter for monitoring the energy consumption of the main components of a computer. In Section VI, we outline several experiments to profile the inference energy efficiency of LLMs with our energy meter and present their results. Section VII discusses how our observations extrapolate to other deployment environments and presents recommendations to improve the energy efficiency of LLMs inference. We present our conclusions in Section VIII.

II. BACKGROUND

Energy consumption is measured in joules (J) or watts per hour (Wh), being 1 Wh = 3600 J, which corresponds to one joule per second, i.e., 1 W = 1 J/s. It is straightforward to measure the total energy consumption of a physical device through a power meter or power monitor as long as there is physical access to the device. However, measuring the energy consumption of software is a far more complicated task:

- A) Many times software runs on servers or machines that cannot be accessed, e.g., on the cloud. Additionally, the software might be running on a virtualized instance, using a fraction of, or multiple physical servers at once. In these cases, it is impossible to use a power meter.
- B) Even if there is physical access to the computer where the software runs, separating the energy usage between its different hardware components, e.g., CPU, memory, GPU, hard disk, is impossible unless individual power monitors are connected to each of these devices and synchronise them to measure the consumption while the software runs.
- C) Modern computers are multi-core and run several processes in a parallel or semi-parallel way. In order to accurately estimate the energy consumption of a particular piece of software, we need to keep track of the exact time at which the software executes, usually to a precision of milliseconds.

Several software tools have emerged to deal with these limitations, allowing the measurement of energy consumption

of applications without the need for any additional hardware. Nonetheless, they still face multiple challenges and constraints that limit their usage:

- D) They depend on low-level libraries and sensors, and are usually constrained by the hardware architecture in which they run. E.g., Intel Running Average Power Limit (RAPL) [17] measures the energy consumption through sensors on the vendor's processors that are not available on other architectures. Additionally, these low-level dependencies are only accessible on bare metal instances due to their high coupling with specific hardware and are not compatible with virtualized instances.
- E) These software tools are generally constrained by the platforms on which they run, such as the operating system. E.g., Kubernetes-based Efficient Power Level Exporter (Kepler) [18] is a tool for measuring the energy consumption of Kubernetes clusters and cannot be used on other settings; Scaphandre [19], a tool to measure the energy consumption of any software, works on Linux and has limited functionality on Windows, with no support for Mac.
- F) For ML models such as LLMs, things get more complicated as libraries take advantage of hardware acceleration, e.g., running on GPUs. Multiple software tools are needed to measure the energy consumption of each device. For instance, RAPL only provides measurements for CPU and (sometimes) memory, *nvidia-smi* [20] only measures the energy consumption of NVIDIA GPUs.

For these reasons energy consumption is ignored by software developers and users.

III. RELATED WORK

This section presents several approaches to facilitate the measurement of energy consumption and the environmental impact of software.

Intel Running Power Average Limit (RAPL) [17] reports the energy consumed (in joules) by different power domains of Intel processors during an interval of time. RAPL's measurements have shown a high accuracy, with an error lower than 2% on the Haswell architecture through different experiments [21]. The latest AMD processors implement a semi-compatible RAPL interface; however, the power domains used by AMD differ from those of Intel and their measurements are not as accurate [22]. *pyRAPL* [22] allows the access to RAPL metrics in Python. NVIDIA NVML is a C-based API for monitoring and managing various states of NVIDIA GPU devices [23]. The NVIDIA CLI, '*nvidia-smi*' [20], provides access to the current power draw in watts. NVML can be easily accessed from Python through *pyNVML* [24]. Even though RAPL and the NVIDIA CLI provide useful and accurate energy readings, their measurements are not provided for individual processes (challenge C).

Scaphandre [19] is a Rust tool that estimates the energy consumption of individual processes by tracking the amount of time that the processor spends on a given process and combining it with RAPL measurements. Scaphandre exports these metrics to Prometheus [25] and, when deployed on a bare metal instance with virtual instances running on top, is capable of propagating energy metrics from the bare metal to the virtual instances to estimate their energy consumption. Even though Scaphandre successfully deals with challenges A, B, C, it still requires to be deployed on the bare metal host to propagate the metrics to the virtual instances. In addition, it only provides metrics for CPU and memory.

Kubernetes-based Efficient Power

Level Exporter (Kepler) [18] uses eBPF [26] to probe CPU performance counters and Linux kernel tracepoints to estimate the energy consumption of Kubernetes clusters, exporting them to Prometheus. Kepler utilizes RAPL and *nvidia-smi* when available; otherwise, it estimates the energy usage of CPU, GPU and memory by combining eBPF probes and pretrained linear regression models. Kepler is limited to Kubernetes clusters (challenge F.) CodeCarbon [27] is a Python package that estimates the total CO₂ emissions of a computer by tracking its energy consumption and combining it with the carbon intensity of geographic regions. It tracks the energy consumption of the CPU, memory, and GPU, using RAPL and *nvidia-smi* if available, otherwise utilizing linear regression models to estimate their consumption. CodeCarbon does not individualize the energy consumed by individual processes (challenge C) and it has issues that affect its accuracy for measuring memory energy consumption [28].

Paper [29] is, to the best of our knowledge, the first to focus on measuring, reporting, and reducing the energy consumption of ML models. The paper analyzes the energy consumption of CNNs, commonly used for computer vision tasks, with a maximum size of 200 million parameters. This paper was published one year before the first Transformer neural network architecture was published [30]. Since then, the growth in the number of parameters in ML models and their energy consumption have dramatically accelerated [5].

Article [31] presents a software framework to measure the total energy consumption of ML experiments. It uses metrics obtained from NVML and RAPL to calculate the energy consumption of the CPU and the GPU, and estimates the corresponding carbon emissions. This framework is used to run several experiments on the energy usage of convolutional neural networks (CNNs) and reinforcement learning agents.

Paper [11] provides a detailed analysis of the environmental impact of training LLMs. It estimates that training BERT [32] – a transformer-based model with 110 million parameters – is roughly equivalent to a transcontinental flight. When neural architecture search is used, the CO₂ footprint grows more than 300 times. Similarly, [8] analyzes the energy consumption and the carbon footprint generated for training transformer architectures. Among other recommendations, they propose to place the training in carbon-free or low-carbon areas to significantly reduce the associated

CO₂ emissions. Paper [10] proposes the use of power-capping to significantly reduce the energy consumption of training LLMs while maintaining accuracy and marginally increasing training time.

While providing insights for reducing the energy consumption and consequent environmental cost of LLMs during training, these works ignore the inference phase, which as we have discussed, has a considerably higher environmental impact. Supporting our claim, Amazon Web Services (AWS) has estimated that 90% of the infrastructure costs associated with using ML in production are devoted to inference while only 10% to training [33].

Some recent research papers also consider the energy consumption of LLMs during inference. Article [34] analyzes the environmental impact of data, experimentation, training, and inference of ML models used for language translation, recommendation and ranking at Meta. The results show a drastic reduction of CO₂ emissions derived from the energy usage of these models by leveraging multiple software and hardware optimizations. Paper [35] surveys numerous optimization techniques for transformers, measuring their effects in terms of inference time and energy, and comparing these effects to the ones seen on CNNs. They apply neural architectural search and hardware co-design to optimize the inference time and energy consumption of a transformer-based architecture, similar to BERT or GPT-2, achieving an improvement of 88x in energy consumption.

In this article we analyze how energy consumption is impacted by the model architecture, batch size and quantization on different sizes of open-source LLM architectures.

IV. MEASURING THE ENERGY CONSUMPTION OF LLMs

After considering the challenges of measuring the energy consumption of software and the limitations of current approaches, we have designed and implemented an energy-measuring Python module, *EnergyMeter*, for measuring the energy consumption of state-of-the-art LLMs. Notably, our approach is the first to the best of our knowledge to measure the energy consumption of CPU, memory, GPU, and storage in environments with Intel processors, NVIDIA GPUs, and Linux operating systems. Consequently, our approach overcomes challenges *A*, *B*, *E*, and *F* while we mitigate challenges *C* and *D* with our deployment environment.

In the following, we describe the design of *EnergyMeter*, the particular methods used to estimate the energy consumption of CPU, GPU, memory, and storage, and its limitations.

A. ENERGYSMETER

EnergyMeter is a Python module that combines RAPL, NVIDIA-SMI, and eBPF to estimate the energy consumption of the main components of a computer with only three lines of code. *EnergyMeter* is open source software and is available at <https://github.com/maufadel/EnergyMeter>. The user initializes an `energy_meter` object that will be in charge of the

energy measurement. The energy measurement starts when the method `energy_meter.begin()` is called. The invocation to the method `energy_meter.end()` stops the measurements and provides the *EnergyMeter* metrics: the total joules used by the system, its distribution in the different components (total joules used by the CPU, memory, GPU, and storage), and the duration of the measurement period in seconds.

In our tests, *EnergyMeter* adds no perceptible overhead while the measurements are taken. There is only a small constant overhead of around 300 ms to calculate the energy metrics after the measurement finishes.

EnergyMeter implements a combination of some of the previously described tools to obtain the energy consumption of the following components:

1) CPU

We leverage pyRAPL to estimate the total energy used by the CPU. We use RAPL for two reasons. First, it has proven to be accurate on the latest generations of Intel processors, thanks to specific sensors used for energy measurements. Secondly, it adds no overhead to the task being measured, as it only queries the number of joules used by the CPU before and after running the task of interest.

As discussed in Section III, RAPL also comes with the drawback of being fully compatible only with Intel processors and requires to be run on bare metal instances. Consequently, this is also a limitation of *EnergyMeter*.

2) GPU

We use pyNVML [24] to access nvidia-smi and sample the power draw of the GPU during the measurement period. The sampling interval is set by default to 500ms and it can be defined by the user to other values. During the measurement period, we run a thread that polls the GPU utilization percentage and its power draw via pyNVML at every specified sampling interval (e.g., every 500 ms) and stores these samples in memory. Once the user invokes the `energy_meter.end()` method, the total energy used by the GPU is calculated as the mean power draw ($\mu_{P_{GPU}}$) applied along the duration of the measurement period (T) using the formula:

$$E_{GPU} = \int_0^T \mu_{P_{GPU}}(t) dt \quad (1)$$

The user can decide if the energy used when the GPU is idle should be accounted for. By default, the energy used when idle is ignored; to do this, we remove the power draw samples when the GPU utilization percentage was 0 from the calculation of $\mu_{P_{GPU}}$ and reduce the time interval $(0, T)$ accordingly.

3) MEMORY

RAPL provides measurements for the DRAM sockets on Intel processors, so we leverage once again pyRAPL to measure the energy consumed by the system memory.

4) STORAGE

EnergyMeter is the first tool to our knowledge to measure the energy consumption of the storage on Linux. We have implemented an approach identical to Joulemeter [36] (a Microsoft tool only compatible with Windows) that calculates the energy used by the storage based on the amount of time it was active and, optionally, idle:

$$E_{HD} = t_{\text{active}}e_{\text{active}} + t_{\text{idle}}e_{\text{idle}} \quad (2)$$

where

t_{active} : time (seconds) the storage was active,
 t_{idle} : time (seconds) the storage was idle,
 e_{active} : energy used by the storage (joules) while active, as defined in its specification,
 e_{idle} : energy used by the storage (joules) while idle, as defined in its specification.

Since it is not possible to directly observe the state of the storage in common operating systems, we calculate the time that the storage was active by tracking the storage IO. To this end, we leverage *eBPF* through custom *bpfftrace* probes that track the total bytes read and written during an interval of time. Once the measurement period finishes, we estimate the total time that the storage was active and replace this in Equation 2 to calculate the total energy consumed:

$$E_{HD} = \frac{b_{\text{read}}}{s_{\text{read}}} + \frac{b_{\text{written}}}{s_{\text{write}}}e_{\text{active}} + t_{\text{idle}}e_{\text{idle}} \quad (3)$$

where

b_{read} : total bytes read during the measurement period,
 s_{read} : storage reading speed in bytes per second,
 b_{written} : total bytes written during the measurement period,
 s_{written} : storage writing speed in bytes per second.

Due to the tools used for estimating energy consumption, *EnergyMeter* has some limitations:

- **Hardware requirements:** *EnergyMeter* is based on RAPL and NVIDIA NVML therefore, it is only compatible with Intel processors and NVIDIA GPUs. RAPL provides energy consumption estimations on some AMD processors but in this case, the measurement is less accurate [22].
- **Software requirements:** *EnergyMeter* uses *eBPF* to access low-level metrics for the storage and is based on Linux systems. Additionally, *EnergyMeter* needs to be run on bare metal instances to have access to RAPL metrics.

In addition, *EnergyMeter* does not provide metrics for individual processes yet and this is considered future work.

V. MEASURING AND IMPROVING THE ENERGY EFFICIENCY OF LLMS WITH ENERGYSMETER

This section describes the scenarios and parameters used for the evaluation of the energy consumption of LLMs during inference, finding opportunities to improve their energy and computing efficiency. We analyze how the energy

consumption of LLMs varies with the following parameters and settings:

- Model size.** The size of an ML model is the number of free parameters that need to be fit to the data. This defines the number of operations that need to be calculated during training and inference as well as how much memory is required to load the model. Recent LLMs reach staggering model sizes. We will analyze how the energy consumption varies according to the model size during the inference phase.
- Model architecture.** While two models might have the same number of parameters, they could still have different architectures, e.g., different embedding sizes, different number of layers, or different number of attention heads. We will analyze if these parameters also affect energy consumption.
- Batch size.** Increasing the batch size increases memory usage while reducing the inference (and training) latency, thanks to the parallelization of operations to be performed. This also impacts the energy usage of LLMs.
- Quantization.** Quantization reduces the memory footprint and processing needed for inference (and training) of models by changing the data type of parameters in the model, e.g., from FP16 to INT8. Even though the accuracy of the model is also affected, the performance decrease is often marginal. We will evaluate how quantization to different data types affects the energy used by LLMs.
- Energy usage of a typical Question Answering (QA) application.** Several organizations are implementing Retrieval QA or ChatBot applications using frameworks that enable the fast development of LLM-based applications. We will implement a QA application and observe its energy consumption during setup and QA.

VI. ENERGY AND PERFORMANCE EVALUATION

The goal of the evaluation is to measure how model size, architecture, batch size, and weights quantization affect the energy consumption and latency of LLMs inference for text generation. For that purpose, we implement the above-described experiments using HuggingFace Transformers [37], a popular Python library that provides access to several pre-trained LLMs, and monitor their performance using *EnergyMeter*. For each experiment, we report the inference energy consumption in joules and the inference latency in milliseconds. We further specify the energy consumption per component for CPU, GPU, memory and storage measured by *EnergyMeter*.

We evaluate the following pre-trained model families from HuggingFace's hub:

- 1) **Pythia** [38]: a suite of 8 different sizes of Transformer models trained on the same public data ranging in size from 70M to 12B parameters.
- 2) **Bloom** [39]: a Transformer language model, essentially similar to GPT-3 (auto-regressive model for next token prediction), trained on the ROOTS corpus [40],

TABLE 2. Main architectural characteristics of the LLMs used in the evaluation.

Model Name	Parameters	Layers	Embedding Dim.	Attention Heads	Vocab. size
EleutherAI/Pythia-70M	70,426,624	6	512	8	50,304
EleutherAI/Pythia-160M	162,322,944	12	768	12	50,304
EleutherAI/Pythia-410M	405,334,016	24	1,024	16	50,304
EleutherAI/Pythia-1B	1,011,781,632	16	2,048	32	50,304
EleutherAI/Pythia-1.4B	1,414,647,808	24	2,048	32	50,304
databricks/Dolly-v2-3b	2,775,086,080	32	2,560	32	50,280
EleutherAI/Pythia-2.8B	2,775,208,960	32	2,560	32	50,304
togethercomputer/RedPajama-INCITE-Base-3B-v1	2,775,864,320	32	2,560	32	50,432
bigscience/bloom-3b	3,002,557,440	30	2,560	32	250,880
openlm-research/open_llama_3b_v2	3,426,473,600	26	2,048	32	32,000
meta-llama/Llama-2-7b-chat-hf (& TheBloke/Llama-2-7B-GGUF)	6,738,415,616	32	4,096	32	32,000
databricks/Dolly-v2-7b	6,856,056,832	32	4,096	32	50,280
EleutherAI/Pythia-6.9B	6,857,302,016	32	4,096	32	50,432
togethercomputer/RedPajama-INCITE-7B-Base	6,857,302,016	32	4,096	32	50,432
bigscience/bloom-7b1	7,069,016,064	32	4,096	32	250,880

comprising hundreds of sources in 46 natural and 13 programming languages. There are different versions of the model ranging from 560M to 176B parameters.

- 3) **Dolly v2** [41]: a language model based on the EleutherAI Pythia family and fine-tuned on a custom, high-quality human-generated instruction dataset, crowdsourced among Databricks employees. The largest Dolly v2 model has 12B parameters and there are smaller versions, also based on Pythia and fine-tuned on the Dolly dataset. Even though Dolly is based on Pythia, the architecture is not identical in the input and output layers as it uses a different vocabulary size.
- 4) **RedPajama** [42]: another family of models based on Pythia with 7B and 3B parameters, fine-tuned on a dataset with the same name as the model, with over 100B text documents coming from 84 CommonCrawl snapshots. Similar to Dolly, RedPajama's architecture is not identical to Pythia. It has a different vocabulary size and disables a mechanism for the parallel calculation of residuals for the models' attention heads.
- 5) **LLaMA 2**: the second version of LLaMA (Large Language Model Meta AI), a state-of-the-art LLM published by Meta with a custom architecture and trained on 1 trillion tokens. We use LLaMA 2 in three different variants (Llama 2 [43], Open LLaMA 2 [44], and a quantized set of Llama models by "TheBloke" [45].)

The specific models and selected architecture characteristics are shown in Table 2.

We have run all the experiments on a single node equipped with Intel(R) Core(TM) i9-9900K @ 3.60GHz, 32 GB RAM, a single NVIDIA GeForce RTX 4090 (24GB RAM) and 1TB SSD (Samsung 970 EVO). This provides a consistent setting for our experiments and ensures EnergyMeter will obtain accurate metrics from RAPL (RAPL is not accurate on AMD processors [22]). In addition, to have storage measurements as accurate as possible, we benchmark the storage writing and reading rate using Flexible IO tester [46] and utilize the

values for energy used when active and idle states reported by AnandTech [47].

A. MODEL SIZE

One of the main differences between the different open-source LLMs is their size in number of parameters. Even though larger models are usually more accurate, they also require more computing resources, which affects energy consumption. To measure how the energy consumption of LLMs varies with their size, we have run an experiment with models from the Pythia suite [38]. This is a suite of LLMs with different sizes and the same base architecture, which has been created to analyze how size affects model performance. We tested 7 models of the Pythia suite with up to 6.9B parameters due to the memory size of the GPU.

The standard procedure for querying LLMs with Hugging Face Transformers is the following. We instantiate a model object and a tokenizer object indicating the model name, e.g., *EleutherAI/pythia-70m*. In our tests we utilize the default configuration provided for each model unless stated otherwise. The model object is a wrapper around the neural network (in our case the PyTorch module) that offers methods for loading, saving the model and its weights, and running inference with a given input. It is possible to load the model on main memory or on the GPU memory; in our case, we utilize the GPU memory. The tokenizer, on the other hand, is always stored in main memory and is an object that converts a string representing text in natural language into a sequence of tokens that can be used as input for the model. A token can be a word, subword, character or symbol, e.g., the tokenizer used by Pythia-70M converts the string "*It's 7 degrees outside.*" into [1147, 434, 818, 7759, 3345, 15], where 1147 corresponds to *It*, 434 to *'s*, 818 to *7*, 7759 to *degrees*, 3345 to *outside*, and 15 to the period. Given a natural language text, we use the tokenizer to convert the text into a sequence of tokens and then feed it to the model object calling the `generate(input)` method to run inference on the neural network. The exact implementation of the `generate(input)` method varies from model to model

but in its basic form it feeds the full sequence of tokens to the neural network to generate a new token, and then, the new token is incorporated at the end of the input and this is fed to the neural network again to generate the next token. This continues until a certain stopping criterion is met, e.g., a maximum number of tokens have been generated or the neural network outputs a special token to indicate the end of the text. The result is a new text formed by the concatenation of the text given as input followed by the text generated by the model.

In this experiment, we load the weights of each model on the GPU memory in 16 bits (i.e., float16) and its tokenizer on the main memory. We then use the tokenizer to convert a short text sampled from The Pile [48], an 825 GB English text corpus used to train several LLMs, to a sequence of tokens and feed it to the model through the `generate(input)` method. For the generation, we set the maximum number of tokens for the output to be 100, which includes the original input tokens that in our case, are 8. In most cases, the output is shorter than 100 tokens because the generation is stopped when the model outputs an end-of-text token. We repeat this process 10 times for each model using the same input text and monitor the energy consumption using *EnergyMeter*. Note that even though the input is always the same, the output of the model is not the same because the text generation is a stochastic process where each next word is sampled from a probability distribution over all words. Because the number of tokens generated in each sample varies, we calculate the energy and latency (execution time) required per token by dividing the total energy used to generate the output divided in the number of tokens generated. Figure 1 presents the energy consumed per token for different model sizes and the execution time.

We can observe that the energy consumed per token and latency grow as the model size increases, except for Pythia-410M (410 million parameters) which uses roughly the same energy than the larger Pythia-1B model (1 billion parameters) and even has a longer inference latency. In addition, we observe that the main energy consumer is the GPU and its energy consumption grows linearly with the number of parameters. The CPU is the second energy consumer and while its consumption also grows linearly, its growth rate is notably slower. The energy consumption of the memory (DRAM) is close to 0 and is constant for all model sizes.

The number of parameters define the number of operations that need to be executed to generate each token so, it is reasonable that for most models, the larger their size, the higher their energy consumption and inference latency. However, this logic does not apply when going from Pythia-410M to Pythia-1B. As described in PaLM [49], we can approximate the number of operations executed to generate each token as twice the number of parameters. So, we see that for Pythia-410M around 820M operations are executed in 19.93ms while for Pythia-1B approximately 2B operations are executed in 16.19ms. Both models have the same base architecture, so why are

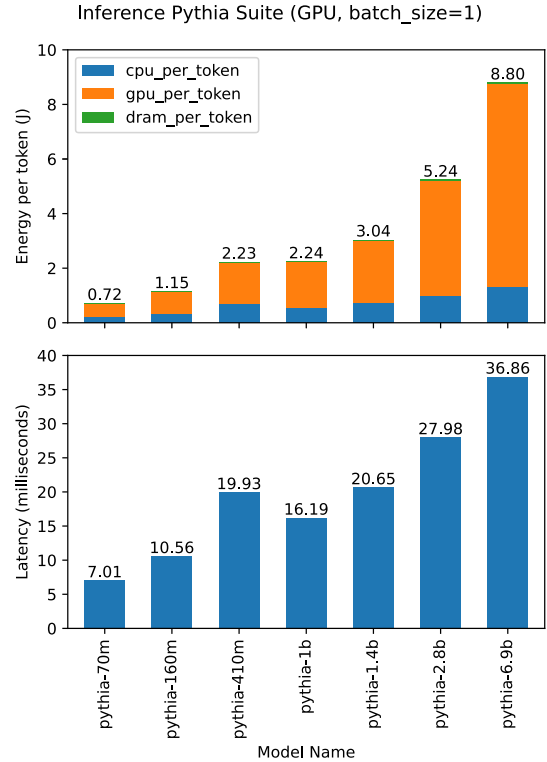


FIGURE 1. Energy consumption and inference latency per token for each model of the Pythia suite from 70M upto 6.9B parameters. Batch size = 1.

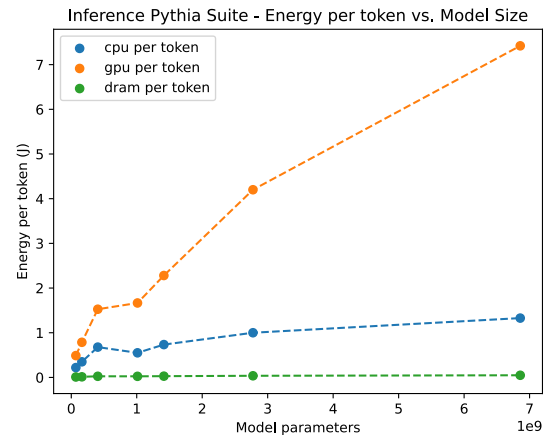


FIGURE 2. Energy consumption per token for CPU, GPU and, memory (DRAM) for models from Pythia suite from 70M to 6.9B parameters, batch size= 1.

the operations of Pythia-1B executed faster than those of Pythia-410M?

Apart from their size, the only difference between these models is that Pythia-410M has 24 layers while Pythia-1B has 16 layers. Usually, deeper models, i.e., models with a higher number of deep layers like Pythia-410M, limit the number of operations that can be parallelized as compared to broader models, i.e., models with a lower number of layers but a higher number of neurons per layer like Pythia-1B. This is because all of the operations of layer n need to be fully executed before being able to execute any of the

operations in layer $n + 1$ as the output of layer n is the input for layer $n + 1$. In practice, this means that when executing the operations of Pythia-1B, the GPU can execute more operations in parallel than for Pythia-410M, resulting in a shorter inference latency. Even though Pythia-1B is faster, both models consume almost the same energy per token, which means the energy consumption per unit of time of Pythia-1B is higher. This is because the GPU parallelizes the operations in more cores, requiring more energy per second.

We observe that even though latency and energy consumption are highly correlated, they are not perfectly correlated: Pythia-2.8B consumes more than 37% more energy than Pythia-1.4B but its latency is only 3.5% higher, and a similar situation happens with Pythia-410M and Pythia-1.4B. Moreover, there is a sub-linear relationship between the number of parameters and the energy consumed for inference. However, this relationship does not depend only on the number of parameters but also seems to be related to other architectural features like the number of layers. We analyze this further in the following experiments.

B. MODEL ARCHITECTURE

In this experiment, we analyze how the model architecture influences the inference time and energy consumption of LLMs. In particular, we will look at the number of layers, the parallelization of layers (e.g., the attention and feedforward layers in Pythia), the storing of attention weights, and finally, the reduction of key/query scores needed to be calculated thanks to the chosen modelling task in Llama v2. For that purpose, we use two groups of different models that have roughly the same number of parameters but different architectures:

- 1) Models with 3B parameters: Pythia-2.8B, Bloom-3B, Open LLaMA v2 3B, Dolly v2 3B, and RedPajama-INCITE-3B.
- 2) Models with 7B parameters: Pythia-6.9B, Bloom-7B, Open Llama v2 7B, Dolly v2 7B, and RedPajama-INCITE-7B.

We load each of the models on the GPU using float16, query each model 10 times with the same input as for the previous experiment, and measure how the energy is distributed among CPU, GPU, and memory per token produced.

1) MODELS WITH 3B PARAMETERS

Even though all models in this experiment are called “3B” models, their sizes range from 2.77B to 3.4B parameters. Figure 3 shows that there are significant differences in energy consumption and latency among the tested models. Bloom 3B is the most efficient one, both in terms of energy and latency (even though it is the second largest model in terms of parameters) while RedPajama-INCITE-3B was the least efficient one. The energy consumption of RedPajama-INCITE-3B is 47% higher than that of Bloom 3B, and this is even more extreme for latency, where

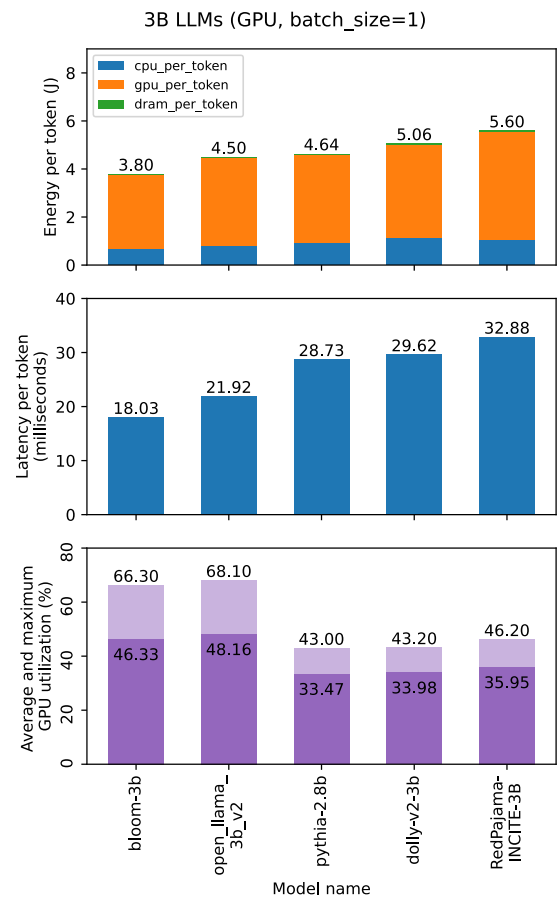


FIGURE 3. Energy used, inference latency per token, and GPU utilization with a batch size of 1 for 3B models.

RedPajama-INCITE-3B has a latency 83% higher than Bloom’s.

Bloom 3B and Open LLaMA 3B v2 are the two most efficient models and share a characteristic: they both have a higher GPU utilization percentage than the other models (46.33% and 48.16% respectively), meaning they have a higher parallelization than Pythia 2.8B, Dolly v2 3B, and RedPajama. This higher parallelization of Bloom 3B and Open LLaMA v2 is achieved in part thanks to having fewer layers than the other models. Bloom 3B has 30 layers and LLaMA 3B 26, while Pythia-2.8B, Dolly v2 3B, and RedPajama have 32 layers.

However, there are other factors in play, after all, Bloom 3B with 30 layers is still 15% more energy and latency efficient than Open LLaMA 3B with 26 layers. If we look at the total number of parameters in Table 2, Bloom 3B has roughly 10% less parameters than Open LLaMA 3B, which is translated in 10% less operations that need to be calculated during inference time. This explains why Bloom 3B requires less energy and achieves a lower inference latency than Open LLaMA 3B.

Interestingly, even though Dolly v2 and RedPajama-INCITE-3B-Base are based on the architecture of Pythia-2.8B, the energy consumption and inference latency between

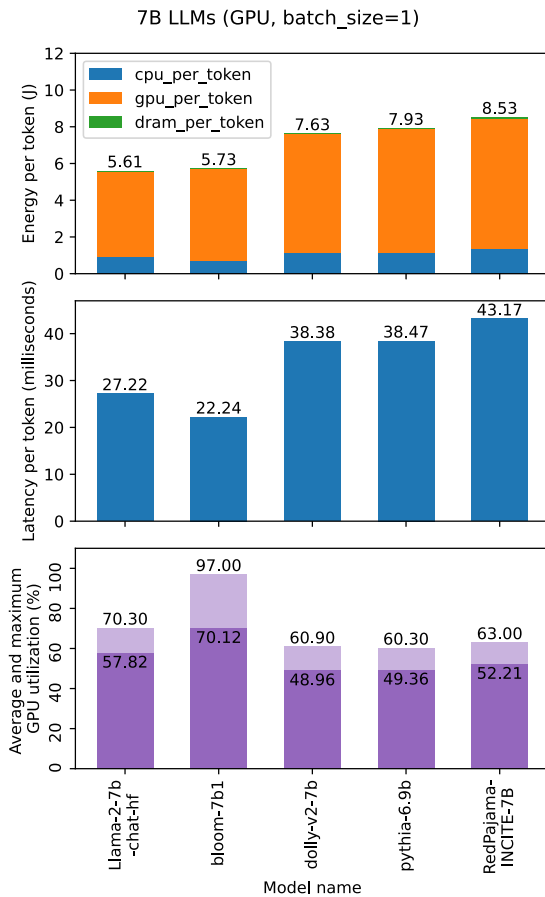


FIGURE 4. Energy used, inference latency per token, and GPU utilization with a batch size of 1 for different models with 7B parameters.

them differ by 10% and 11%, respectively. The Pythia suite uses a technique introduced with GPT-J [50], that places the attention and feedforward layers in parallel to improve its throughput by about 15%. RedPajama-INCITE-3B has disabled this mechanism and therefore the attention and feedforward layers are not run in parallel, which explains its slower performance compared to the other two models.

2) MODELS WITH 7B PARAMETERS

The behaviour is similar for models with 7B parameters but the differences among them are even higher than among the models with 3B parameters. In particular, the energy consumption increases up to 53% from the most efficient and the least efficient model (for the 3B models this was 47%), and latency of the slowest model is almost double the one of the slowest model (for the 3B models this was 83%).

There is also an important difference with the previous case. Llama v2 7B is the most efficient model in terms of energy consumption but, not in latency, where Bloom 7B is the fastest. This shows that even though there is a high correlation between latency and energy consumption, latency cannot be directly taken as a measurement of energy consumption. Llama implements several improvements for its training such as not storing the attention weights and not

computing the key/query scores that are masked due to the causal nature of the language modelling task [43], which also reduces its inference time.

C. BATCH SIZE

The batch size defines the number of samples (i.e., prompts in the case of LLMs) that are processed at once in the neural network and is among the most commonly tuned parameters in ML because of its great impact on training and inference times. In this experiment, we evaluate how the batch size affects inference time and energy consumption per token in LLMs. The procedure for the test is similar to the one used in the previous experiments: loading the models on the GPU with FP16 and using short prompts (less than 10-tokens long) randomly sampled from The Pile to match the desired batch size.

By increasing the batch size during inference, the inference time per token decreases thanks to the parallelization of operations. However, this comes at the cost of memory (main memory or GPU memory). The maximum batch size possible depends on the memory available in the specific hardware used. Even though energy consumption is highly correlated with latency, the higher utilization of the GPU also increases the power draw per time unit.

Figure 5 shows the relative and total energy consumed per token as the batch size increases from 1 to 100 for models of the Pythia suite. We see that the energy consumption decreases when the batch size increases for Pythia models of all sizes. However, the reduction in energy consumption is greater for small models, reaching up to 20x for Pythia-70m while “only” 3x for Pythia-6.9B. We also observe that the inference latency per token drops significantly for all models when their batch size increases (Figure 5). Again, this has a greater impact on smaller models: on Pythia-70m, the inference is 50x faster by increasing the batch size from 1 to 100 prompts, while on Pythia-6.9B this is improved 7x.

The latency is reduced more aggressively than the energy consumption because the GPU requires more power to run a higher number of operations in parallel when the batch size is larger, i.e., the higher the utilization, the higher the power draw per time unit. In addition, the benefits of increasing the batch size are less important for large models because the number of parallel operations executed in the GPU reach the limit at lower batch sizes; from that point, even though more operations could be run in parallel, this is not possible because all GPU cores are busy. We observe this in the GPU percentage utilization recorded during the experiments, shown in Figure 5.

We repeat the experiment with the group of models with 7B parameters to analyze how different architectures interact with a changing number of prompts per batch. Figure 6 shows the results. We can observe that in all cases, a larger batch size reduces the energy consumption per token just like for the Pythia family. The proportion of energy and inference time reduced by using a larger batch size varies across different architectures: Llama v2 7B has the greatest

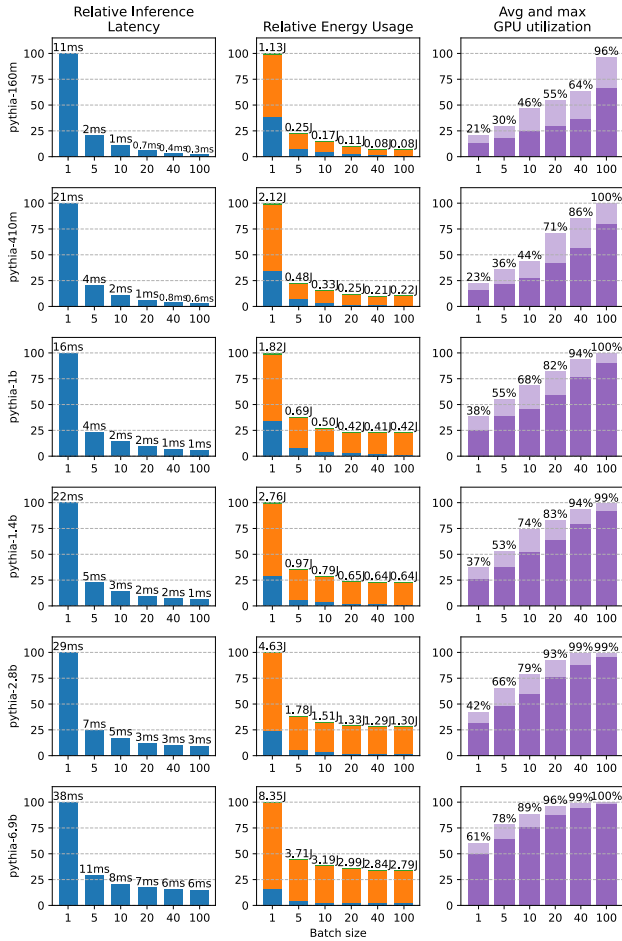


FIGURE 5. Relative and total energy per token for different batch sizes with models of the Pythia suite. 100% latency and energy correspond to a batch size equal to 1.

benefit reducing the latency per token up to 95% when using a batch size of 20 instead of 1. As explained in the previous experiment, Llama v2 7B implements several optimizations in its architecture that make it achieve the best energy and latency performance when increasing its batch size.

In contrast, Bloom 7B shows the least benefit when increasing its batch size, ending up with a similar performance to RedPajama-INCITE-BASE-7B when both use a batch size of 20. We can see why in the GPU utilization percentage: when Bloom 7B uses a batch size of 5, its GPU utilization is 99% so, increasing the batch size from that point on shows no improvement. This shows that there is a plateau in the benefits obtained by increasing the batch size that is reached when the GPU utilization is close to 100%, at this point increasing the batch size does not offer any reduction in latency or energy consumption. Instead, RedPajama-INCITE-BASE-7B has a low parallelization of operations as explained in Subsection VI-B, therefore, it is necessary to increase the batch size to higher values than for other models to take full advantage of the GPU parallelization of operations.

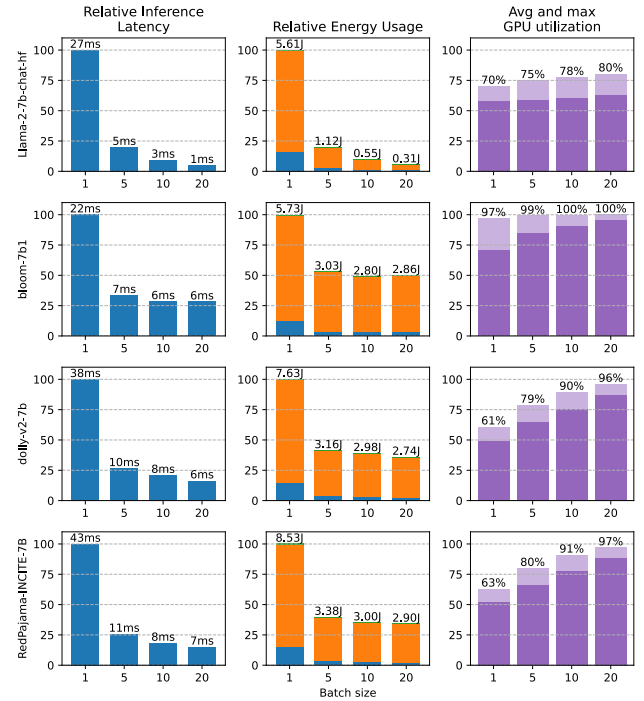


FIGURE 6. Relative latency and energy per token, and average and max GPU utilization for different batch sizes and 7B parameters models. 100% latency and energy correspond to a batch size equal to 1.

D. QUANTIZATION

HuggingFace offers the possibility to load any model in 8- and 4-bits to memory using bitsandbytes [51]. This reduces the memory footprint by 4x for 8 bits as compared to using 32 bits, and up to 8x when using 4 bits. However, due to the implementation details of bitsandbytes, inference latency increases when using quantized models with a precision lower than 16 bits [52], which also increases the energy consumption in our tests.

Another option to run quantized models is to use models whose weights have already been quantized. We found a model repository that has quantized the weights of Llama v2 7B to 8, 5, 4, 3 and 2-bit versions in GGUF format [45]. This format has been recently created to support quantization of models and can be run on ctransformers, a port of transformers that uses C++ and GGML as its backend machine learning framework. We have tested these versions and the original Llama v2 7B from HuggingFace (run with the original transformers library with its backend on Pytorch).

Quantization not only decreases the memory footprint of the models but also their inference energy usage and latency (Figure 7). Latency always decreases from 16 bits to 2 bits, while the energy consumption savings seem to reach their efficiency peak at 4 bits (after that the energy consumption remains almost constant).

E. ENERGY USAGE OF A TYPICAL QUESTIONS AND ANSWERS APPLICATION

We have implemented a QA application using LangChain [53], one of the most popular frameworks for the

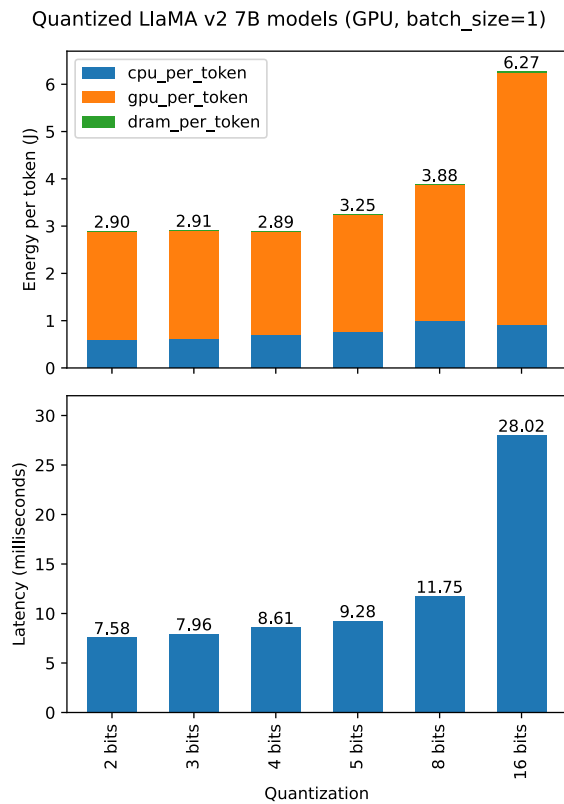


FIGURE 7. Energy consumption and latency per token for different quantization configurations of the Llama v2 7B model (meta-llama/Llama-2-7b-chat-hf for 16 bits and TheBloke/Llama-2-7B-GGUF for all others).

development of LLM-based applications, Transformers [54], and ChromaDB [55]. Our application follows the pipeline proposed for QA applications by LangChain:

- 1) **Loading:** loading texts from unstructured files, in our case PDF files published on Arxiv during August 2023, extracted from the Arxiv dataset [56]. This dataset contains 18870 files with a total size of 58GB.
- 2) **Splitting:** the text is split into chunks of 1000 characters with an overlap of 200 characters within chunks. For this operation, we use the ‘RecursiveCharacterTextSplitter’ from LangChain, which splits the texts into paragraphs or sentences with the given character length.
- 3) **Storage:** the chunks are converted to 384-dimensional vectors using the model “all-MiniLM-L6-v2” from HuggingFace Transformers (loaded on the GPU), which is optimized for text similarity. These vectors are then stored in the vector store ChromaDB [55].
- 4) **Retrieval:** when the user types a question, the application queries the vector store, which finds the most similar chunks of text to the question using Euclidean distance.
- 5) **Generation:** the user question and the n most similar text chunks are given to the LLM Llama-2-7b-chat-hf, which is in charge of generating an answer.

Application Setup - Energy Distribution

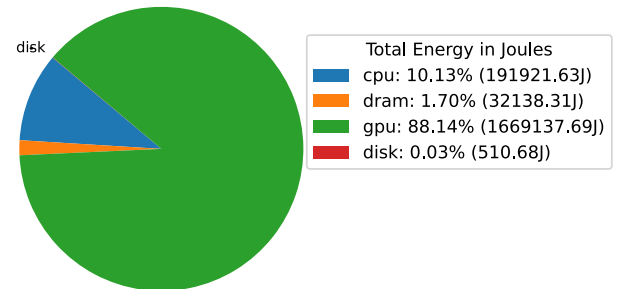


FIGURE 8. Total energy used during vector database creation and its distribution among the different components for our QA Application that leverages LangChain, Transformers and ChromaDB.

We separate this pipeline into two stages: the first three steps, which we will call the *setup* stage, are performed only the first time the application is launched and may be re-run for eventual updates. This stage is expected to be run much less frequently than the second stage, called the *QA* stage, which will be run for answering the user’s questions. The setup stage is expected to be more energy-intensive and computing-demanding than the QA stage because it needs to load, preprocess, vectorize and store a large dataset. However, the QA stage will be run much more frequently and it includes querying an LLM, so it is even more important to consider its energy and computing demand.

We run the setup stage in our instance and use the EnergyMeter to monitor it. This is a compute-intensive task as 58GB of text needs to be preprocessed, embedded, and finally stored in the vector store. The whole stage took 6 hours and 20 minutes and used a total of 1.9 million joules or 526 Wh. In Figure 8, we can see the main energy consumption comes from the GPU, concentrating 86% of all the energy used during this stage. Apart from the GPU, the CPU uses around 11%, the memory less than 2% and the storage (disk in this case) only 0.03% even though the operation reads more than 58GB and creates the database with a size of 15GB.

Once we have run the setup stage, the QA stage starts. A user asks a question and the application will reply using the knowledge base (i.e., the vector store) and an LLM. As we have seen previously, the most efficient LLM was Llama v2 7B, so we utilize this model for our application. We utilize a batch size of 1 and measure the energy consumption for answering 10 questions we have manually generated based on a random sample of ten papers contained in the dataset. The questions are the following:

- 1) What is the bowtie framework?
- 2) What is a non-Hermitian system?
- 3) How can continual learning improve complex facial expression recognition?
- 4) How can we increase the kilohertz sensitivity of a gravitational-wave interferometer?
- 5) Is it possible to forecast the dynamics of BTC with Markov Chains?
- 6) What are metal nanoclusters?

- 7) What's the hierarchical NAS framework (HNAS-Reg)?
- 8) What optimization techniques can you use in deep learning?
- 9) What's a Haldane model?
- 10) What's retrieval augmentation?

The energy consumption for querying the database and answering a question with this LLM is shown in Figure 9. The component that utilizes the most energy is the GPU, which is needed to run inference on the LLM. On the other extreme, the disk uses less than 0.01% of the total energy. This is because the index used for queries in ChromaDB is loaded in memory, so it is only necessary to read the four most similar chunks to the query from the disk. These chunks are of 1000 characters (defined in step 2) so, their size is roughly 1kB each.

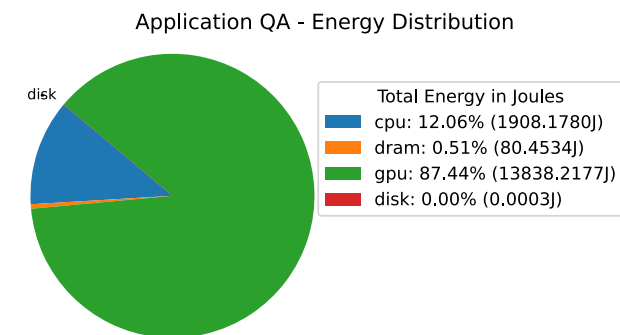


FIGURE 9. Total energy for answering 10 questions with batch_size=1 with meta-llama/llama-2-7b-chat-hf from HuggingFace.

Most of the energy consumed by the QA application in both stages is attributed to the LLM. It becomes evident that we have to focus on the LLM component to improve the energy efficiency of the whole application.

VII. DISCUSSION

Throughout this work, we have intentionally ignored the accuracy of the LLMs we have tested. We have done this because of different reasons. First, in most works, the focus is put on the accuracy of the models and how to improve them while largely ignoring our topic of interest: their energy consumption. Secondly, there are several metrics to measure the accuracy of LLMs depending on the task they need to solve, so we believe it is better to perform an accuracy evaluation when a specific use case is considered; energy consumption is use case agnostic instead. In addition, increasing the batch size during inference only alters the number of operations run in parallel so this technique only affects energy consumption and inference time, not the generated text or its accuracy. Finally, quantization of the weights in models has been shown to have detrimental effects on their accuracy [57], but how much the accuracy decreases depends on several factors including the specific model used, the use case, and the evaluation metric.

Moreover, the precise distribution of energy usage among CPU, memory, GPU, and storage will vary in other machines depending on the specific hardware they utilize. However, we do not expect this distribution to greatly vary; GPU should

always remain as the most energy-consuming component when using LLMs, followed by the CPU. In addition, in our measurements we are ignoring the energy consumed by other components such as the cooling of the machine. This is usually measured as a by-product of the energy used for computing, e.g., by multiplying the energy used for computing by the Power Usage Effectiveness (PUE) ratio, so it is possible to estimate the total energy used by only monitoring the energy devoted to computing. This means that if we only aim at improving the energy efficiency of the software (instead of obtaining a concrete number of joules or Wh used by the instance where the software runs,) we can ignore the PUE and focus on improving the energy used by the computing.

A. STRATEGY FOR EFFICIENTLY RUNNING INFERENCE WITH LLMs

Leveraging the insights obtained during the development of this work, we have put together a simple strategy for running inference with LLMs efficiently, both in terms of energy and latency (i.e., throughput). The strategy is as follows:

- 1) **Select the most efficient LLM for your task.** There are several LLMs available on online platforms such as Hugging Face, published with the accuracy scores they have achieved for different tasks, e.g., on the Hugging Face LLM Leaderboard [58]. It is important to choose the best model not only based on its accuracy score for the task of interest, but also the most efficient one by considering its energy efficiency and inference latency. For this, define a minimum accuracy score threshold and from all the models with a higher score than the threshold, choose the one with the smallest number of parameters. Even though more parameters usually translate into a better accuracy, this is not always the case (e.g., Pythia-12B has an accuracy of 39.59 on ARC, a set of grade-school science questions [59], while the smaller Llama-2-7b-chat-hf has an accuracy of 53.07 on the same test [58].) If there are multiple models with the same accuracy score and the same size, use the model with the lowest number of layers as this reduces inference latency and energy consumption during inference.
- 2) **If available, use a quantized version of the model.** This saves memory and has the potential to improve its energy efficiency up to 2x and reducing its inference latency more than 3x.
- 3) **Increase the batch size until reaching a GPU utilization close to 100% or until the memory is full.** In Experiment VI-C, we have shown that increasing the batch size reduces the energy consumption for all model families and sizes up to 20x. For interactive applications with a dynamic number of requests, it is possible to batch several requests in time windows, e.g., all requests in a window of X seconds are put in a batch. In this way, an LLM can serve multiple concurrent requests

with a sub-linear growth of its energy consumption and inference latency per request.

VIII. CONCLUSION

We presented *EnergyMeter*, a software-based approach for measuring the energy consumption of different hardware components, and evaluated the different parameters that affect the energy consumption during inference of LLMs.

EnergyMeter is an open-source Python tool that simplifies the measurement of the total energy used by any software or code chunk, providing the energy consumed by the CPU, memory, GPU, and storage. To the best of our knowledge, *EnergyMeter* is the first tool to measure the energy consumption of storage on Linux systems thanks to its use of eBPF. *EnergyMeter* does not add any significant overhead while the measurements are taken. However, *EnergyMeter* has two limitations: (1) it requires Intel processors and NVIDIA GPUs, and (2) it requires Linux and bare metal systems. This last drawback is the most significant as many applications run on virtualized cloud instances but as discussed in Section VI, in many cases it is possible to profile the application on a local deployment using *EnergyMeter* before deploying it on the virtualized cloud instance.

Moreover, we have used *EnergyMeter* to profile the energy consumption of several state-of-the-art LLMs. Through several experiments, we have obtained the following insights:

- It is possible to monitor the energy consumption of software with purely software-based approaches. Even though the accuracy of the approaches varies depending on the component to be monitored and the specific hardware used, the measurements provide valuable information to optimize energy efficiency.
- Latency is not enough to estimate the energy consumption of LLMs. In Section VI, we have observed that inference latency and energy consumption are correlated but in some cases, a model might have a higher energy consumption and a lower inference latency than another model.
- Increasing the number of parameters of LLMs increases inference latency and energy consumption sub-linearly. Energy consumption grows at a higher rate than latency because the parallelization of the model operations reduces latency but increases energy consumption per time unit.
- Model architecture plays an essential role in the ability of parallelizing model operations. Models with the same number of parameters and a higher parallelization capability are more efficient in terms of both their inference latency and energy consumption.
- Increasing the batch size for LLMs significantly reduces the energy used per token. The energy savings stop once the GPU is fully utilized, as it is not possible to parallelize operations any further.
- While quantization effectively reduces the memory footprint, its energy consumption and latency does not always decrease with higher levels of quantization. This

depends heavily on the quantization implementation and there is a point at which the energy consumption does not decrease anymore.

We hope this work will contribute to the scientific community and the industry to increase the energy efficiency of software, and in particular LLMs, reducing their associated environmental impact. In the future, we plan to work in the development of software for dynamically modifying the configuration of LLMs so their energy consumption, inference time and resource usage are automatically optimized.

REFERENCES

- [1] *Concise: The New Way to Read News*. Accessed: Aug. 3, 2023. [Online]. Available: <https://www.concise.app>
- [2] *Introducing Microsoft 365 Copilot: Your Copilot for Work*. Accessed: Aug. 3, 2023. [Online]. Available: <https://blogs.microsoft.com/blog/2023/03/16/introducing-microsoft-365-copilot-your-copilot-for-work/>
- [3] A. Borji, "Generated faces in the wild: Quantitative comparison of stable diffusion, midjourney and DALL-E 2," 2022, *arXiv:2210.00586*.
- [4] H. Ko, S. Lee, Y. Park, and A. Choi, "A survey of recommendation systems: Recommendation models, techniques, and application fields," *Electronics*, vol. 11, no. 1, p. 141, Jan. 2022.
- [5] P. Villalobos, J. Sevilla, T. Besiroglu, L. Heim, A. Ho, and M. Hobbhahn, "Machine learning model sizes and the parameter gap," 2022, *arXiv:2207.02852*.
- [6] W. Xin Zhao et al., "A survey of large language models," 2023, *arXiv:2303.18223*.
- [7] *Gpt-4 Architecture, Datasets, Costs and More Leaked*. Accessed: Aug. 3, 2023. [Online]. Available: <https://the-decoder.com/gpt-4-architecture-datasets-costs-and-more-leaked/>
- [8] D. Patterson, J. Gonzalez, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. So, M. Texier, and J. Dean, "Carbon emissions and large neural network training," 2021, *arXiv:2104.10350*.
- [9] U.S. Energy Information Admin. *Electricity Consumption by Country*. Accessed: Aug. 3, 2023. [Online]. Available: <https://www.eia.gov/international/data/world/electricity/electricity-consumption>
- [10] J. McDonald, B. Li, N. Frey, D. Tiwari, V. Gadeppally, and S. Samsi, "Great power, great responsibility: Recommendations for reducing energy for training language models," 2022, *arXiv:2205.09646*.
- [11] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for modern deep learning research," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, 2020, pp. 13693–13696.
- [12] Hugging Face. *Displaying Carbon Emissions for Your Model*. Accessed: Aug 8, 2023. [Online]. Available: <https://huggingface.co/docs/hub/model-cards-co2>
- [13] *Chatgpt Sets Record for Fastest-Growing User Base: Analyst Note*. Accessed: Aug. 3, 2023. [Online]. Available: <https://www.reuters.com/technology/chatgpt-sets-record-fastest-growing-user-base-analyst-note-2023-02-01/>
- [14] A. Sasha Luccioni, S. Viguier, and A.-L. Ligozat, "Estimating the carbon footprint of Bloom, a 176B parameter language model," 2022, *arXiv:2211.02001*.
- [15] OpenAI. *ChatGPT: Get Instant Answers, Find Creative Inspiration, and Learn Something New*. Accessed: Aug. 3, 2023. [Online]. Available: <https://openai.com/chatgpt>
- [16] *Introducing Microsoft 365 Copilot—Your Copilot for Work*, Microsoft, Redmond, WA, USA, Mar. 2023.
- [17] H. David, E. Gorbato, U. R. Hanebutte, R. Khanna, and C. Le, "RAPL: Memory power estimation and capping," in *Proc. ACM/IEEE Int. Symp. Low-Power Electron. Design (ISLPED)*, Aug. 2010, pp. 189–194.
- [18] Kepler. (2022). *Sustainable Computing IO*. [Online]. Available: <https://github.com/sustainable-computing-io/kepler>
- [19] B. Petit, "Scaphandre," Version V1.0, 2023. [Online]. Available: <https://github.com/hubblo-org/scaphandre>
- [20] *Nvidia System Management Interface*. Accessed: Aug. 8, 2023. [Online]. Available: <https://developer.nvidia.com/nvidia-system-management-interface>
- [21] K. N. Khan, M. Hirki, T. Niemi, J. K. Nurminen, and Z. Ou, "RAPL in action: Experiences in using RAPL for power measurements," *ACM Trans. Model. Perform. Eval. Comput. Syst.*, vol. 3, no. 2, pp. 1–26, Jun. 2018.

- [22] R. Schöne, T. Ilsche, M. Biele, M. Velten, M. Schmidl, and D. Hackenberg, "Energy efficiency aspects of the AMD Zen 2 architecture," in *Proc. IEEE Int. Conf. Cluster Comput. (CLUSTER)*, Sep. 2021, pp. 562–571.
- [23] (2021). *NVIDIA Developer Nvidia Management Library (NVML)*. Accessed: Aug. 10, 2023. [Online]. Available: <https://developer.nvidia.com/management-library-nvml>
- [24] A. Benoit, A. Nouredine, and R. Rouvoy. (2023). *PyNVML: Python Bindings To the Nvidia Management Library*. Accessed: Aug. 10, 2023. [Online]. Available: <https://pypi.org/project/pynvml/>
- [25] B. Rabenstein and J. Volz, "Prometheus: A next-generation monitoring system (Talk)," 2015.
- [26] A. Starovoitov and D. Borkmann, *EBPF: A Technology for Dynamic Kernel Programming*. Linux Foundation. [Online]. Available: <https://ebpf.io/>.
- [27] B. Courty et al., "MLCO2/codecarbon: V2.4.1," Zenodo, Version v2.4.1, May 2024, doi: [10.5281/zenodo.11171501](https://doi.org/10.5281/zenodo.11171501).
- [28] *MLCO2 and BCG GAMMA Codecarbon: Methodology*. Accessed: Aug. 9, 2023. [Online]. Available: <https://mlco2.github.io/codecarbon/methodology.html>
- [29] D. Li, X. Chen, M. Becchi, and Z. Zong, "Evaluating the energy efficiency of deep convolutional neural networks on CPUs and GPUs," in *Proc. IEEE Int. Conf. Big Data Cloud Comput. (BDCLOUD), Social Comput. Netw. (SocialCom), Sustain. Comput. Commun. (SustainCom) (BDCLOUD-SocialCom-SustainCom)*, Oct. 2016, pp. 477–484.
- [30] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 1–19.
- [31] P. Henderson, J. Hu, J. Romoff, E. Brunsell, D. Jurafsky, and J. Pineau, "Towards the systematic reporting of the energy and carbon footprints of machine learning," *J. Mach. Learn. Res.*, vol. 21, no. 1, pp. 10039–10081, 2020.
- [32] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.
- [33] G. Hutt, V. Viswanathan, and A. Nadolski, "Deliver high performance ml inference with aws inferentia," 2019.
- [34] C.-J. Wu, "Sustainable AI: Environmental implications, challenges and opportunities," *Proc. Mach. Learn. Syst.*, vol. 4, pp. 795–813, Aug. 2022.
- [35] S. Kim, C. Hooper, T. Wattanawong, M. Kang, R. Yan, H. Genc, G. Dinh, Q. Huang, K. Keutzer, M. W. Mahoney, Y. S. Shao, and A. Gholami, "Full stack optimization of transformer inference: A survey," 2023, *arXiv:2302.14017*.
- [36] A. Kansal, F. Zhao, J. Liu, N. Kothari, and A. A. Bhattacharya, "Virtual machine power metering and provisioning," in *Proc. 1st ACM Symp. Cloud Comput.*, Jun. 2010, pp. 39–50.
- [37] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in *Proc. Conf. Empirical Methods Natural Lang. Processing: Syst. Demonstrations*, 2020, pp. 38–45.
- [38] S. Biderman, H. Schoelkopf, G. Q. Anthony, H. Bradley, K. O'Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff, A. Skowron, L. Sutawika, and O. Van Der Wal, "Pythia: A suite for analyzing large language models across training and scaling," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 2397–2430.
- [39] B. Workshop et al., "Bloom: A 176B-parameter open-access multilingual language model," 2022, *arXiv:2211.05100*.
- [40] H. Laurençon, "The bigscience roots corpus: A 1.6TB composite multilingual dataset," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 31809–31826.
- [41] M. Conover, M. Hayes, A. Mathur, J. Xie, J. Wan, S. Shah, A. Ghodsi, P. Wendell, M. Zaharia, and R. Xin, "Free dolly: Introducing the world's first truly open instruction-tuned llm," *Company Blog Databricks*, Apr. 2023. Accessed: Nov. 2023. [Online]. Available: <https://www.databricks.com/blog/2023/04/12/dolly-first-open-commercially-viable-instr-uction-tuned-llm>
- [42] Together AI. (2021). *Redpajama Models V1*. [Online]. Available: <https://www.together.ai/blog/redpajama-models-v1>
- [43] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.
- [44] X. Geng and H. Liu. (2023). *Openllama: An Open Reproduction of Llama*. [Online]. Available: https://github.com/openlm-research/open_llama
- [45] TheBloke, "Llama-2-7B-GGUF," Hugging Face, 2023. Accessed: Nov. 12, 2023. [Online]. Available: <https://huggingface.co/TheBloke/Llama-2-7B-GGUF>
- [46] J. Axboe. (2021). *FIO: Flexible I/O Tester*, 2021. Accessed: Nov. 30, 2023. [Online]. Available: <https://fio.readthedocs.io/en/latest/>
- [47] B. Tallis, "The Samsung 970 EVO SSD review: The 64-layer refresh," *AnandTech*, Apr. 2018. Accessed: Aug. 20, 2023. [Online]. Available: <https://www.anandtech.com/show/12670/the-samsung-970-evo-ssd-review>
- [48] L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Pang, H. He, A. Thite, N. Nabeshima, S. Presser, and C. Leahy, "The pile: An 800GB dataset of diverse text for language modeling," 2021, *arXiv:2101.00027*.
- [49] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann, and P. Schuh, "PaLM: Scaling language modeling with pathways," *J. Mach. Learn. Res.*, vol. 24, no. 240, pp. 1–113, 2023.
- [50] B. Wang and A. Komatsuzaki. (2021). *GPT-J-6B: A 6 Billion Parameter Autoregressive Language Model*. [Online]. Available: <https://arankomatsuzaki.wordpress.com/2021/06/04/gpt-j/>
- [51] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: Efficient finetuning of quantized LLMs," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2024.
- [52] T. Dettmers. (2021). *How to Use Bitsandbytes With Pytorch*. [Online]. Available: <https://github.com/TimDettmers/bitsandbytes/issues/6>
- [53] H. Chase, "LangChain," Oct. 2022. [Online]. Available: <https://github.com/langchain-ai/langchain>
- [54] T. Wolf, "Transformers: State-of-the-art natural language processing," in *Proc. Conf. Empirical Methods Natural Lang. Process., Syst. Demonstrations*. Association for Computational Linguistics, Oct. 2020, pp. 38–45. [Online]. Available: <https://www.aclweb.org/anthology/2020.emnlp-demos.6>
- [55] C. Core, "Chroma: The AI-native open-source embedding database," Version 0.4.10. Accessed: Aug. 31, 2021. [Online]. Available: <https://github.com/chroma-core/chroma>
- [56] *Arxiv Dataset*. Kaggle, 2023. [Online]. Available: <https://www.kaggle.com/dsv/6443346>, doi: [10.34740/KAGGLE/DSV/6443346](https://doi.org/10.34740/KAGGLE/DSV/6443346).
- [57] T. Dettmers and L. Zettlemoyer, "The case for 4-bit precision: K-bit inference scaling laws," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 7750–7774.
- [58] E. Beeching, C. Fourrier, N. Habib, S. Han, N. Lambert, N. Rajani, O. Sanseviero, L. Tunstall, and T. Wolf. (2023). *Open Llm Leaderboard*. [Online]. Available: https://huggingface.co/spaces/HuggingFaceH4/open_llm_leaderboard
- [59] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, "Think you have solved question answering? Try ARC, the AI2 reasoning challenge," 2018, *arXiv:1803.05457*.



MAURICIO FADEL ARGERICH received the degree in information systems engineering from Universidad Tecnológica Nacional, Argentina, in 2014, and the M.S. degree in data science from the Sapienza University of Rome, Italy, in 2018. He is currently pursuing the Ph.D. degree in software, systems, and computing with Universidad Politécnica de Madrid, Spain. In 2013, he received the Summer Research Scholarship from The University of Auckland, New Zealand,

to carry out a research project in automatic link generation as a Visiting Student. He has multiple years of experience as a Software Engineer and a Researcher. He has published his work at multiple international IoT and AI conferences and symposia. His current research interest includes leveraging AI algorithms for optimizing the energy efficiency of distributed applications on edge and cloud.



MARTA PATIÑO-MARTÍNEZ is currently the Director of the Distributed Systems Laboratory and a Professor with Universidad Politécnica de Madrid. She is also coordinating the AI4PublicPolicy European Project delivering an Open Cloud platform for automated, scalable, transparent, and citizen-centric policy management based on AI technologies. She has published several papers on these topics in international conferences and journals. Her research interests include data management, distributed systems, scalability, and fault-tolerant systems. She has served on the program committee of several conferences, including VLDB, ICDCS, ICDE, middleware, SRDS, and EDCC.

...